

Digital Observatory — AI Prompting Strategy & Prompt Templates (v1)

1. AI Development Philosophy

This document defines:

- AI collaboration workflows
- prompt engineering standards
- architectural constraint enforcement
- generation safety rules
- debugging methodologies
- validation systems
- incremental implementation strategy

The goal is:

controlled AI-assisted engineering.

Not:

chaotic code generation.

Core Philosophy

AI is:

- an implementation accelerator
- a component generator
- a refactor assistant
- a debugging collaborator
- a documentation tool

AI is NOT:

- the system architect
- the interaction designer
- the observability strategist
- the performance authority

The human remains responsible for:

- architecture
- system integrity
- performance governance
- emotional tone
- interaction coherence

2. Prompting Rules

RULE 1

Never ask AI to build:

"the portfolio"

Always isolate systems.

RULE 2

Every prompt must define:

Requirement	Purpose
system scope	prevent chaos
architectural context	preserve consistency
state ownership	avoid conflicts
performance expectations	maintain responsiveness
styling constraints	visual cohesion
accessibility requirements	usability

RULE 3

Prompt for:

behaviors.

Not visuals alone.

RULE 4

AI-generated code must always be:

- reviewed
- validated
- profiled
- integrated incrementally

Never blindly merged.

3. Universal Prompt Template

Project Context:

You are building a module for the Digital Observatory portfolio.

Stack:

- Next.js
- TypeScript
- Tailwind CSS
- Zustand
- Framer Motion

Design Language:

- restrained
- observability-focused
- atmospheric
- industrial precision
- performance-aware

Goal:

[DEFINE SPECIFIC GOAL]

Component Scope:

[DEFINE RESPONSIBILITIES]

Constraints:

- No duplicated systems
- Reuse shared components
- Preserve accessibility
- Avoid unnecessary motion

- Maintain observatory tone

State Rules:

[LOCAL VS GLOBAL STATE]

Performance Rules:

- minimize rerenders
- lazy-load heavy systems
- isolate expensive rendering

Deliverables:

- component structure
- interaction logic
- accessibility support
- styling
- state handling

4. UI Generation Templates

Window Component Prompt

Build a reusable observatory Window component.

Requirements:

- draggable
- resizable
- focus-aware
- keyboard accessible
- smooth weighted transitions

Architecture Constraints:

- integrate with centralized WindowState
- local resize state only
- shared motion tokens
- no duplicated window logic

Visual Constraints:

- layered dark surfaces
- restrained glow
- subtle translucency
- compact technical typography

Dock Prompt

Build a mac-inspired observatory dock.

Requirements:

- hover magnification
- active app indicators
- smooth transitions
- keyboard accessibility

Constraints:

- restrained animation
- observatory visual language
- performance-aware rendering
- centralized animation config

5. State Management Prompting

State Prompt Structure

Always define:

- state owner
- event source
- rerender triggers
- persistence rules
- recovery behavior

GOOD State Prompt

Build draggable observatory windows using centralized WindowState for focus management and local component state for resizing interactions.

BAD State Prompt

Add draggable windows.

A terrifyingly effective method for generating future suffering.

6. Motion & Animation Prompting

Animations must communicate:

- hierarchy
- runtime state
- focus
- interaction feedback

NOT:

- random spectacle
-

Motion Prompt Template

Add subtle weighted transitions that reinforce spatial continuity without distracting from observability content.

Motion Constraints

Constraint	Purpose
short durations	responsiveness
restrained easing	professionalism
meaningful transitions	clarity
reduced-motion support	accessibility

7. Performance Prompting

Performance Review Prompt

Analyze this component tree for:

- unnecessary rerenders

- animation overhead
- heavy rendering paths
- memory risks
- state over-coupling

Suggest optimizations without changing architecture.

Rendering Constraints

Always specify:

Requirement	Purpose
lazy loading	startup speed
isolated rendering	responsiveness
memoization strategy	rerender control
adaptive complexity	scalability

8. Debugging Prompting

Debug Prompt Template

Problem:
Window focus breaks after minimizing and restoring.

Expected Behavior:
Restored window should remain top-most.

Architecture:

- centralized WindowState
- local resize state
- global z-index management

Find likely causes and propose minimal fixes.

Debugging Rules

Always provide:

- reproduction steps
- expected behavior
- current behavior
- architectural context
- recent changes

Without context AI debugging becomes:

divination with syntax highlighting.

9. Refactor Prompting

Refactor Prompt Template

```
Refactor this component while:  
- preserving behavior  
- improving readability  
- reducing rerenders  
- maintaining observatory styling  
- avoiding architectural changes
```

NEVER Prompt

```
Improve everything.
```

That sentence has probably generated thousands of unnecessary abstraction layers.

10. Validation Prompting

Every generated system must undergo validation.

Validation Prompt

Review this implementation for:

- architectural consistency
- performance risks
- accessibility issues
- observatory design compliance
- duplicated logic
- unnecessary complexity

Validation Categories

Area	Questions
Architecture	duplicated systems?
State	ownership clear?
Motion	meaningful?
Performance	rerender risks?
Accessibility	keyboard usable?
Design	observatory-consistent?

11. Architecture Enforcement Prompts

Enforcement Prompt

Verify this implementation against the Digital Observatory architecture.

Check:

- shared component reuse
- centralized state usage
- design token compliance
- motion consistency
- rendering isolation
- accessibility support

12. Context Injection Templates

Large projects fail when AI loses context.

Standard Context Injection

```
This system belongs to the Digital Observatory portfolio.

The observatory uses:
- restrained industrial design
- observability-focused interactions
- centralized state systems
- performance-aware rendering
- atmospheric but minimal motion
```

Window System Context

```
This component integrates with:
- centralized WindowManager
- shared motion tokens
- observatory surface hierarchy
- accessibility layer
```

13. AI Anti-Patterns

Dangerous Prompts

Bad Prompt	Why It Fails
build my whole portfolio	architectural chaos
make it futuristic	visual inconsistency
add cool animations	meaningless motion
improve everything	uncontrolled refactors
optimize this	vague performance goals

Dangerous Behaviors

Behavior	Risk
merging huge AI outputs blindly	instability
regenerating systems repeatedly	architecture drift
changing libraries impulsively	integration chaos
skipping validation	hidden regressions

14. Incremental Generation Workflow

Correct Workflow

```
Define Scope
  ↓
Generate Small System
  ↓
Review Architecture
  ↓
Test Isolation
  ↓
Integrate Carefully
  ↓
Validate
  ↓
Profile Performance
```

Wrong Workflow

```
Generate everything.
Hope reality cooperates.
```

Reality rarely does.

15. Component Build Pipelines

Recommended Build Order

1. Shared primitives
2. Layout systems
3. Navigation systems
4. Window systems
5. Content apps
6. Engine Room
7. Terminal
8. Engineer systems
9. Hidden systems
10. Polish

Per-Component Pipeline

- Architecture
- Prompt
 - Generate
 - Review
 - Validate
 - Integrate
 - Optimize

16. AI Review Checklists

UI Checklist

- consistent spacing
- observatory styling
- restrained motion
- accessibility support
- responsive layout

State Checklist

- ownership clear
 - no duplicated truth
 - rerenders isolated
 - persistence intentional
 - recovery paths defined
-

Performance Checklist

- lazy loading used
 - heavy rendering isolated
 - animations optimized
 - no render storms
 - motion scalable
-

17. Recovery & Rollback Prompting

Recovery Prompt

Design a recovery strategy for this observatory subsystem.

Requirements:

- safe reset support
 - fallback rendering
 - recoverable failures
 - preserved accessibility
 - minimal user disruption
-

Rollback Strategy

Always keep:

- stable checkpoints
- isolated branches
- modular commits
- reversible integrations

Because eventually:

one experimental feature will attempt murder.

18. Production Readiness Validation

Pre-Launch Prompt

Audit this system for production readiness.

Review:

- performance
- accessibility
- responsiveness
- browser compatibility
- architectural consistency
- rendering stability
- error handling

19. Core AI Collaboration Principles

PRINCIPLE 1

AI should accelerate engineering.

Not replace thinking.

PRINCIPLE 2

Architecture must remain centralized.

PRINCIPLE 3

Every generated system requires validation.

PRINCIPLE 4

Performance is part of implementation quality.

PRINCIPLE 5

Incremental integration prevents catastrophe.

PRINCIPLE 6

The observatory must feel:

intentionally engineered.

Not randomly generated.